

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет ИУ
Кафедра ИУ5

Курс «Парадигмы и конструкции языков программирования»

Отчет по лабораторной работе №5
«Модульное тестирование в Python»
Вариант №7

Выполнил:
студент группы ИУ5-32Б:
Гаранин Е. Д.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Гапанюк Ю. Е.
Подпись и дата:

Москва, 2025 г.

ОГЛАВЛЕНИЕ

1. Задание для лабораторной работы
2. Листинг кода
3. Пример выполнения

ЗАДАНИЕ ДЛЯ ЛАБОРАТОРНОЙ РАБОТЫ

1. Выберите любой фрагмент кода из лабораторной работы 2.
2. Модифицируйте код таким образом, чтобы он был пригоден для модульного тестирования.
3. Разработайте модульные тесты. В модульных тестах необходимо применить следующие технологии:
 - TDD - фреймворк (не менее 3 тестов).
 - BDD - фреймворк (не менее 3 тестов).

ЛИСТИНГ КОДА

lab_python_opp __init__.py:

lab_python_opp circle.py:

```
from lab_python_opp.figure import Figure
from lab_python_opp.color import FigureColor
import math

class Circle(Figure):
    FIGURE_TYPE = "Круг"

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE

    def __init__(self, color_param, r_param):
        self.r = r_param
        self.fc = FigureColor()
        self.fc.colorproperty = color_param

    def square(self):
        return math.pi*(self.r**2)

    def __repr__(self):
        return '{} {} цвета радиусом {} площадью {}'.format(
            Circle.get_figure_type(),
            self.fc.colorproperty,
            self.r,
            self.square()
        )
```

lab_python_opp color.py:

```
class FigureColor:
    def __init__(self):
        self._color = None

    @property
    def colorproperty(self):
        return self._color

    @colorproperty.setter
    def colorproperty(self, value):
        self._color = value
```

lab_python_opp figure.py:

```
from abc import ABC, abstractmethod

class Figure(ABC):
    @abstractmethod
    def square(self):
        pass
```

lab_python_opp rectangle.py:

```
from lab_python_opp.figure import Figure
from lab_python_opp.color import FigureColor

class Rectangle(Figure):
    FIGURE_TYPE = "Прямоугольник"

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE

    def __init__(self, color_param, width_param, height_param):
        self.width = width_param
        self.height = height_param
        self.fc = FigureColor()
        self.fc.colorproperty = color_param

    def square(self):
        return self.width*self.height

    def __repr__(self):
        return '{} {} цвета шириной {} и высотой {} площадью {}.'.format(
            Rectangle.get_figure_type(),
            self.fc.colorproperty,
            self.width,
            self.height,
            self.square()
        )
```

lab_python_opp square.py:

```
from lab_python_opp.rectangle import Rectangle

class Square(Rectangle):
    FIGURE_TYPE = "Квадрат"

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE

    def __init__(self, color_param, side_param):
        self.side = side_param
        super().__init__(color_param, self.side, self.side)

    def __repr__(self):
        return '{} {} цвета со стороной {} площадью {}.'.format(
            Square.get_figure_type(),
            self.fc.colorproperty,
            self.side,
            self.square()
        )
```

lab_python_opp main.py:

```
from lab_python_opp.rectangle import Rectangle
from lab_python_opp.circle import Circle
from lab_python_opp.square import Square
import numpy

def main():
```

```

N = 6

r = Rectangle("синего", N, N)
c = Circle("зеленого", N)
s = Square("красного", N)

print(r)
print(c)
print(s)

print(f"\nДемонстрация работы внешнего пакета:")
print(f"Версия numru: {numru.__version__}")
print(f"Пример использования numru для вычисления площади круга:
{numru.pi * (N**2)}")

if __name__ == "__main__":
    main()

```

features steps __init__.py:

features steps figure_steps.py:

```

from behave import given, when, then
from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square
import math

FIGURE_CLASSES = {
    "прямоугольник": Rectangle,
    "круг": Circle,
    "квадрат": Square
}

@given('я создаю "{color}" "{figure_type}"')
def step_given_create_figure_base(context, color, figure_type):
    """Шаг: Сохраняет тип фигуры и цвет в контексте."""
    context.figure_type = FIGURE_CLASSES.get(figure_type)
    context.color = color
    assert context.figure_type is not None, f"Неизвестный тип фигуры:
{figure_type}"

@when('его ширина равна {width:d} и высота равна {height:d}')
def step_when_set_rectangle_dims(context, width, height):
    """Шаг: Инициализирует Прямоугольник."""
    assert context.figure_type == Rectangle
    context.figure = context.figure_type(context.color, width, height)

@when('его радиус равен {radius:g}')
def step_when_set_circle_dims(context, radius):
    """Шаг: Инициализирует Круг."""
    assert context.figure_type == Circle
    context.figure = context.figure_type(context.color, radius)

@when('его сторона равна {side:d}')
def step_when_set_square_dims(context, side):
    """Шаг: Инициализирует Квадрат."""
    assert context.figure_type == Square
    context.figure = context.figure_type(context.color, side)

```

```

@then('его площадь должна быть {area:g}')
def step_then_check_area(context, area):
    """Шаг: Проверяет точное значение площади."""
    assert context.figure.square() == area, f"Ожидалось {area}, получено {context.figure.square()}"

@then('его площадь должна быть приблизительно {area:g}')
def step_then_check_area_approx(context, area):
    """Шаг: Проверяет приблизительное значение площади (для float)."""
    assert math.isclose(context.figure.square(), area, rel_tol=1e-5), f"Ожидалось {area}, получено {context.figure.square()}"

@then('он также должен быть "{parent_class_name}"')
def step_then_check_inheritance(context, parent_class_name):
    """Шаг: Проверяет наследование (isinstance)."""
    parent_class = None
    if parent_class_name == "прямоугольником":
        parent_class = Rectangle

    assert parent_class is not None, f"Неизвестный родительский класс: {parent_class_name}"
    assert isinstance(context.figure, parent_class), f"Объект не является экземпляром {parent_class_name}"

```

features figures.feature:

```

# language: ru
Функционал: Вычисление площади геометрических фигур
Как пользователь, я хочу создавать различные фигуры
и корректно вычислять их площадь

Сценарий: Создание Прямоугольника и вычисление его площади
Дано я создаю "синий" "прямоугольник"
Когда его ширина равна 10 и высота равна 20
Тогда его площадь должна быть 200

Сценарий: Создание круга и вычисление его площади
Дано я создаю "зеленый" "круг"
Когда его радиус равен 1
Тогда его площадь должна быть приблизительно 3.14159

Сценарий: Создание Квадрата и проверка типа
Дано я создаю "красный" "квадрат"
Когда его сторона равна 7
Тогда его площадь должна быть 49
И он также должен быть "прямоугольником"

```

test_tdd.py:

```

import pytest
import math
from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

def test_rectangle_square():
    """Тест: Вычисление площади прямоугольника."""
    r = Rectangle("синий", 10, 5)
    assert r.square() == 50

def test_circle_square():

```

```

    """Тест: Вычисление площади круга (сравнение float)."""
    c = Circle("зеленый", 7)
    expected_area = math.pi * (7**2)
    assert c.square() == pytest.approx(expected_area)

def test_square_square_and_type():
    """Тест: Площадь квадрата и проверка наследования."""
    s = Square("красный", 6)
    assert s.square() == 36
    assert isinstance(s, Rectangle)

def test_square_repr():
    """Тест: Строковое представление (__repr__) Квадрата."""
    s = Square("красный", 6)
    expected_repr = "Квадрат красный цвета со стороной 6 площадью 36."
    assert repr(s) == expected_repr

```

ПРИМЕР ВЫПОЛНЕНИЯ

Прямоугольник синего цвета шириной 6 и высотой 6 площадью 36.

Круг зеленого цвета радиусом 6 площадью 113.09733552923255.

Квадрат красного цвета со стороной 6 площадью 36.

Демонстрация работы внешнего пакета:

Версия numru: 2.2.5

Пример использования numru для вычисления площади круга: 113.09733552923255

```
===== test session starts =====
platform win32 -- Python 3.12.5, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\erofe\Documents\Материалы_МГТУ\3_семестр\ПикАП\Лаба_5
plugins: anyio-4.11.0
collected 4 items

test_tdd.py .... [100%]

===== 4 passed in 0.06s =====
```

```
USING RUNNER: behave.runner:Runner
Функционал: Вычисление площади геометрических фигур # features/figures.feature:2
  Как пользователь, я хочу создавать различные фигуры
  и корректно вычислять их площадь
    Тогда его площадь должна быть 200 # features/steps/figure_steps.py:38 0.000s
    Тогда его площадь должна быть 200 # features/steps/figure_steps.py:38
    Тогда его площадь должна быть приблизительно 3.14159 # features/steps/figure_steps.py:43 0.002s
    Тогда его площадь должна быть приблизительно 3.14159 # features/steps/figure_steps.py:43
    И он также должен быть "прямоугольником" # features/steps/figure_steps.py:48 0.000s
    И он также должен быть "прямоугольником" # features/steps/figure_steps.py:48
1 feature passed, 0 failed, 0 skipped
3 scenarios passed, 0 failed, 0 skipped
10 steps passed, 0 failed, 0 skipped
Took 0min 0.004s
```

Вывод: я изучил возможности модульного тестирования в языке Python.